

CSS (Cascading Style Sheets)

1. What is CSS?

CSS (Cascading Style Sheets) is a language used to **style HTML content**.

HTML tells the browser **what** to display.

CSS tells the browser **how** to display it.

CSS controls:

- ⑩ Colors
- ⑩ Fonts
- ⑩ Layout
- ⑩ Spacing
- ⑩ Responsiveness

Without CSS, websites look plain and unorganized.

2. Why Use CSS?

CSS allows:

- ⑩ Separation of content (HTML) from design (CSS)
- ⑩ Easy maintenance
- ⑩ Faster updates
- ⑩ Consistent design across pages
- ⑩ Better user experience

One CSS file can style **multiple HTML pages**.

3. CSS Syntax

Every CSS rule has **three parts**:

```
selector {  
  property: value;  
}
```

Example:

```
p {  
  color: blue;  
  font-size: 16px;  
}
```

⑩ **Selector** → What element to style

⑩ **Property** → What to change

⑩ **Value** → How to change it

4. Ways to Apply CSS

1. Inline CSS (Not Recommended)

Applied directly inside an HTML tag.

```
<p style="color:red;">Hello</p>
```

Hard to maintain

Not reusable

2. Internal CSS (Limited Use)

Written inside <style> in HTML.

```
<style>  
  p {  
    color: blue;  
  }  
</style>
```

Only affects one page

3. External CSS (Best Practice)

CSS written in a separate .css file.

```
<link rel="stylesheet" href="style.css">
```

Clean

Reusable

Professional standard

5. CSS Selectors

Selectors choose **which HTML elements** to style.

1. Element(HTML Tag) Selector

```
p {  
  color: black;  
}
```

Styles all <p> elements.

2. Class Selector

```
.box {  
  background-color: yellow;  
}
```

⑩ Reusable

⑩ Applied using class=""

```
<div class="box"></div>
```

3. ID Selector

```
#header {  
  background-color: gray;  
}
```

⑩ Used once per page

⑩ Applied using id=""

```
<div id="header"></div>
```

4. Group Selector

```
h1, h2, p {  
  font-family: Arial;  
}
```

6. Colors in CSS

CSS supports different color formats.

Color Name

```
color: red;
```

HEX

color: #ff0000;

RGB

color: rgb(255, 0, 0);

RGBA (with opacity)

color: rgba(255, 0, 0, 0.5);

7. Units in CSS

Absolute Units

- ⑩ px – Fixed size

Relative Units

- ⑩ % – Relative to parent
- ⑩ em – Relative to element
- ⑩ rem – Relative to root
- ⑩ vw, vh – Viewport width/height

Best practice: Use rem, %, vh, vw for responsive design.

8. Text & Font Styling

Font Properties

```
p {  
  font-family: Arial, sans-serif;  
  font-size: 16px;  
  font-weight: bold;  
}
```

Text Properties

```
h1 {  
  text-align: center;  
  line-height: 1.5;  
}
```

9. The CSS Box Model (Very Important)

Every HTML element is a **box**.

Box Model Parts:

1. Content
2. Padding – Space inside border
3. Border – Surrounds padding
4. Margin – Space outside element

```
.box {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 10px;  
}
```

Box Sizing

box-sizing: border-box;

Ensures padding doesn't increase width.

10. Display Property

Controls how elements behave.

Value	Description
block	New line, full width
inline	Same line, no width
inline-block	Inline + size
none	Hidden

11. Flexbox (Flexible Box Layout)

Flexbox is one-dimensional

It works in **one direction at a time**:

- ⑩ Either **row (horizontal)**

⑩ Or **column (vertical)**

It is best for:

- ⑩ Navigation bars
- ⑩ Buttons alignment
- ⑩ Centering items
- ⑩ Small layout sections

How Flexbox Works

Step 1: Make a container flex

```
<div class="container">  
  <div class="box">1</div>  
  <div class="box">2</div>  
  <div class="box">3</div>  
</div>
```

```
.container {  
  display: flex;  
}
```

When you add:

display: flex;

All direct children become **flex items**

They automatically align in a **row**

Important Flexbox Properties

1. flex-direction

Controls direction:

flex-direction: row; /* default */

flex-direction: column;

flex-direction: row-reverse;

flex-direction: column-reverse;

2. justify-content

Controls alignment **horizontally** (main axis)

justify-content: center;
justify-content: space-between;
justify-content: space-around;
justify-content: space-evenly;

Example:

```
.container {  
  display: flex;  
  justify-content: center;  
}
```

3. align-items

Controls alignment **vertically**

align-items: center;
align-items: flex-start;
align-items: flex-end;

Example: Perfect Centering

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
}
```

This centers content **both horizontally and vertically**.

2. CSS Grid

Grid is two-dimensional

It controls:

⑩ Rows

⑩ Columns

At the same time.

Best for:

⑩ Full page layouts

⑩ Dashboards

⑩ Complex sections

How Grid Works

Step 1: Make container grid

```
<div class="container">
  <div class="box">1</div>
  <div class="box">2</div>
  <div class="box">3</div>
  <div class="box">4</div>
</div>

.container {
  display: grid;
  grid-template-columns: 200px 200px;
}
```

This creates:

⑩ 2 columns

⑩ Items automatically fill rows

Important Grid Properties

1. grid-template-columns

```
grid-template-columns: 1fr 1fr 1fr;
```

Creates 3 equal columns.

fr = fraction of available space.

2.grid-template-rows

```
grid-template-rows: 100px 200px;
```

Defines row heights.

3. gap

```
gap: 20px;
```

Adds space between grid items.

Example: 3 Equal Columns Layout

```
.container {
  display: grid;
```



```
grid-template-columns: repeat(3, 1fr);
gap: 20px;
}
```

Flexbox vs Grid (Simple Comparison)

Flexbox	Grid
1D layout	2D layout
Best for small components	Best for full layouts
Easier for alignment	More powerful structure control
Content-driven	Layout-driven

13 Responsive Design

Makes websites work on all devices.

Media Queries

```
@media (max-width: 768px) {
  body {
    background-color: lightgray;
  }
}
```

Mobile-first design is recommended.

14 Transitions & Hover Effects

Hover

```
button:hover {
  background-color: blue;
}
```

Transition

```
button {
  transition: background-color 0.3s;
}
```